SCR2043 OPERATING SYSTEMS

Name	: Evelyn Goh Yuan Qi	Marks
Student ID	: A23CS0222	
Section	: 2	

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. **Download necessary source codes:**

   Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. **Compile the source files:**

   Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. **Execute the dummy processes:**

   Run all the dummy processes

```
./mainprocess &
```

   Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

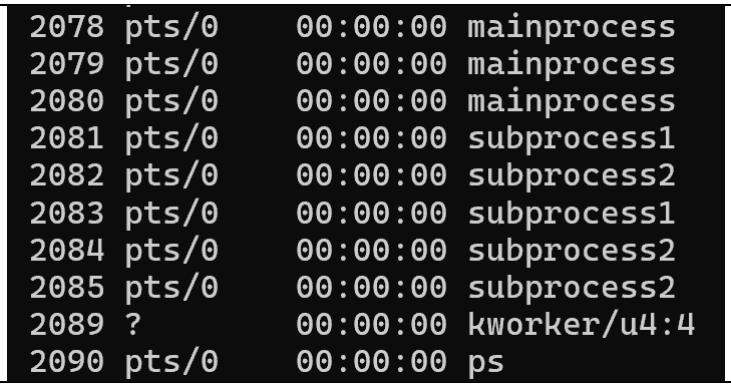
Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

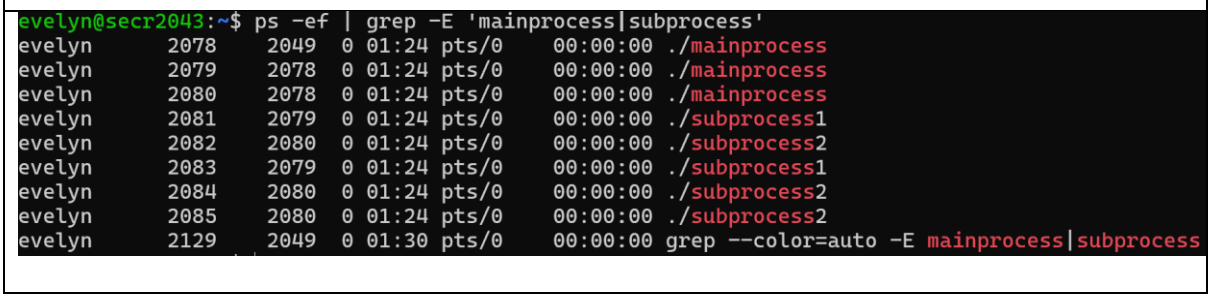
Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command
`ps -e`
Output


```
2078 pts/0    00:00:00 mainprocess
2079 pts/0    00:00:00 mainprocess
2080 pts/0    00:00:00 mainprocess
2081 pts/0    00:00:00 subprocess1
2082 pts/0    00:00:00 subprocess2
2083 pts/0    00:00:00 subprocess1
2084 pts/0    00:00:00 subprocess2
2085 pts/0    00:00:00 subprocess2
2089 ?        00:00:00 kworker/u4:4
2090 pts/0    00:00:00 ps
```

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command
ps -ef | grep -E 'mainprocess|subprocess'
Output


```
evelyn@secr2043:~$ ps -ef | grep -E 'mainprocess|subprocess'
evelyn      2078    2049  0 01:24 pts/0    00:00:00 ./mainprocess
evelyn      2079    2078  0 01:24 pts/0    00:00:00 ./mainprocess
evelyn      2080    2078  0 01:24 pts/0    00:00:00 ./mainprocess
evelyn      2081    2079  0 01:24 pts/0    00:00:00 ./subprocess1
evelyn      2082    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2083    2079  0 01:24 pts/0    00:00:00 ./subprocess1
evelyn      2084    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2085    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2129    2049  0 01:30 pts/0    00:00:00 grep --color=auto -E mainprocess|subprocess
```

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command
ps -ef | grep -E 'subprocess'
Output

```
evelyn@secr2043:~$ ps -ef | grep -E 'subprocess'
evelyn      2081    2079  0 01:24 pts/0    00:00:00 ./subprocess1
evelyn      2082    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2083    2079  0 01:24 pts/0    00:00:00 ./subprocess1
evelyn      2084    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2085    2080  0 01:24 pts/0    00:00:00 ./subprocess2
evelyn      2131    2049  0 01:32 pts/0    00:00:00 grep --color=auto -E subprocess
```

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)
- Command (`cmd`)

Command
`ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess'`
Output

```
evelyn@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd | grep -E 'subprocess
   2081 evelyn    0.0  0.1 ./subprocess1
   2082 evelyn    0.0  0.1 ./subprocess2
   2083 evelyn    0.0  0.1 ./subprocess1
   2084 evelyn    0.0  0.1 ./subprocess2
   2085 evelyn    0.0  0.1 ./subprocess2
   2139 evelyn    0.0  0.1 grep --color=auto -E subprocess
```

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (`pmem`)?

Command
ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'
Output

```
evelyn@secr2043:~$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem | grep -E 'subprocess'
   2081 evelyn    0.0  0.1 ./subprocess1
   2082 evelyn    0.0  0.1 ./subprocess2
   2083 evelyn    0.0  0.1 ./subprocess1
   2084 evelyn    0.0  0.1 ./subprocess2
   2085 evelyn    0.0  0.1 ./subprocess2
   2141 evelyn    0.0  0.1 grep --color=auto -E subprocess
```

Question 6

Construct a command using `ps`, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command
ps --forest -C mainprocess -C subprocess1 -C subprocess2
Output

```
evelyn@secr2043:~$ ps --forest -C mainprocess -C subprocess1 -C subprocess2
    PID TTY          TIME CMD
   2078 pts/0    00:00:00 mainprocess
   2079 pts/0    00:00:00  \_ mainprocess
   2081 pts/0    00:00:00  |   \_ subprocess1
   2083 pts/0    00:00:00  |   \_ subprocess1
   2080 pts/0    00:00:00  \_ mainprocess
   2082 pts/0    00:00:00      \_ subprocess2
   2084 pts/0    00:00:00      \_ subprocess2
   2085 pts/0    00:00:00      \_ subprocess2
```

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
pstree -c -s 2078
Output

```
evelyn@secr2043:~$ pstree -c -s 2078
systemd---sshd---sshd---sshd---bash---mainprocess-+-mainprocess-+-subprocess1
                                                  |             `-subprocess1
                                                  `-mainprocess-+-subprocess2
                                                                |-subprocess2
                                                                `-subprocess2
```

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
sudo renice -5 2081
Output

```
evelyn@secr2043:~$ ps -o pid,nice,comm -C subprocess1
    PID  NI COMMAND
   2081   0 subprocess1
   2083   0 subprocess1
evelyn@secr2043:~$ sudo renice -5 2081
[sudo] password for evelyn:
2081 (process ID) old priority 0, new priority -5
```

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output

```
evelyn@secr2043:~$ killall -15 mainprocess
Main process (ID: 2078) received signal: 15. Terminating...
Main process (ID: 2079) received signal: 15. Terminating...
Main process (ID: 2080) received signal: 15. Terminating...
[1]+  Done                    ./mainprocess
```

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

// Function executed by child process
void child_process() {
    printf("Child process started with PID: %d\n", getpid());

    // Random sleep time between 1 and 5 seconds
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds...\n", sleep_time);
    sleep(sleep_time);

    printf("Child process (PID: %d) finished\n", getpid());
}

int main() {
    // Seed the random number generator
    srand(time(NULL));

    printf("Parent process started with PID: %d\n", getpid());

    // Create child process
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        // Parent process
        printf("Parent process waiting for child (PID: %d)...\n", pid);
        wait(NULL);  // Wait for child to finish
        printf("Parent process resuming and exiting\n");
    } else {
        // Fork failed
        perror("fork failed");
        return 1;
    }

    return 0;
}
```

Output:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

// Function executed by child process
void child_process() {
    printf("Child process started with PID: %d\n", getpid());

    // Random sleep time between 1 and 5 seconds
    int sleep_time = rand() % 5 + 1;
    printf("Child process sleeping for %d seconds...\n", sleep_time);
    sleep(sleep_time);

    printf("Child process (PID: %d) finished\n", getpid());
}

int main() {
    // Seed the random number generator
    srand(time(NULL));

    printf("Parent process started with PID: %d\n", getpid());

    // Create child process
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        child_process();
        exit(0);
    } else if (pid > 0) {
        // Parent process
        printf("Parent process waiting for child (PID: %d)...\n", pid);
        wait(NULL);  // Wait for child to finish
        printf("Parent process resuming and exiting\n");
    } else {
        // Fork failed
        perror("fork failed");
        return 1;
    }

    return 0;
}
```

Output of example.c:

```
evelyn@secr2043:~$ nano example.c
evelyn@secr2043:~$ gcc example.c -o example
evelyn@secr2043:~$ ./example
Parent process started with PID: 2174
Parent process waiting for child (PID: 2175)...
Child process started with PID: 2175
Child process sleeping for 2 seconds...
Child process (PID: 2175) finished
Parent process resuming and exiting
```